

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Machine Learning

ASSESSMENT TOOL 3 – Experiments

Course Code:	MLA0404	Course Title:	Deep Learning
CO Assessed:	CO1	Assessment:	Assessment Tool 3 – Experiments
Weightage:	35% of CIA	Total Marks:	(5 Questions × 10 Marks)
CO1 Statement:	<i>Apply the fundamental concepts of machine learning and neural networks to design and analyze learning algorithms using perceptrons, multilayer perceptrons, forward propagation, backpropagation, gradient descent optimization techniques, and Maximum Likelihood Estimation for solving real-world problems.</i>		
Student Name:	P.Gayathri	Reg. No.:	192425277
Section / Batch:	AIML	Date:	24/07/26

Code of Conduct

I K.AMRUTHA/192425366 (NAME/ REG NO) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: K.Amrutha

1.Implementation of a Single Artificial Neuron

```
import numpy as np

def step_activation(z):
    return 1 if z >= 0 else 0

inputs = np.array([1, 0, 1])
weights = np.array([0.5, -0.3, 0.8])
bias = -0.4

weighted_sum = np.dot(inputs, weights) + bias
output = step_activation(weighted_sum)

print("Inputs:", inputs)
print("Weights:", weights)
print("Bias:", bias)
print("Weighted Sum:", weighted_sum)
print("Output:", output)

print("\nTesting Different Inputs:")

test_inputs = [
    [0, 0, 0],
    [0, 0, 1],
    [0, 1, 0],
    [1, 0, 0],
    [1, 1, 1]
]

for x in test_inputs:
    z = np.dot(x, weights) + bias
    y = step_activation(z)
    print("Input:", x, "Weighted Sum:", z, "Output:", y)
```

Output

```
*** Inputs: [1 0 1]
Weights: [ 0.5 -0.3  0.8]
Bias: -0.4
Weighted Sum: 0.9
Output: 1

Testing Different Inputs:
Input: [0, 0, 0] Weighted Sum: -0.4 Output: 0
Input: [0, 0, 1] Weighted Sum: 0.4 Output: 1
Input: [0, 1, 0] Weighted Sum: -0.7 Output: 0
Input: [1, 0, 0] Weighted Sum: 0.09999999999999998 Output: 1
Input: [1, 1, 1] Weighted Sum: 0.6 Output: 1
```

2. Perceptron for AND Gate

```
[2]
✓ Os
y = np.array([0, 0, 0, 1])

weights = np.zeros(2)
bias = 0
learning_rate = 0.1
epochs = 10

def activation(z):
    return 1 if z >= 0 else 0

for epoch in range(epochs):
    for i in range(len(X)):

        prediction = activation(np.dot(X[i], weights) + bias)
        error = y[i] - prediction

        weights += learning_rate * error * X[i]
        bias += learning_rate * error

    print("Final Weights:", weights)
    print("Final Bias:", bias)

    correct = 0

    print("\nAND Gate Predictions:")

    for i in range(len(X)):
        prediction = activation(np.dot(X[i], weights) + bias)

        print(X[i], "=>", prediction)

        if prediction == y[i]:
            correct += 1

    accuracy = correct / len(y) * 100

    print("\nAccuracy:", accuracy, "%")

*** Final Weights: [0.2 0.1]
Final Bias: -0.20000000000000004

AND Gate Predictions:
[0 0] => 0
[0 1] => 0
[1 0] => 0
[1 1] => 1

Accuracy: 100.0 %
```

3. Perceptron for OR Gate

```
[3]
✓ Os
weights = np.zeros(2)
bias = 0

learning_rate = 0.1
epochs = 10

def activation(z):
    return 1 if z >= 0 else 0

for epoch in range(epochs):
    for i in range(len(X)):
        prediction = activation(np.dot(X[i], weights) + bias)
        error = y[i] - prediction
        weights += learning_rate * error * X[i]
        bias += learning_rate * error

print("Final Weights:", weights)
print("Final Bias:", bias)

correct = 0

print("\nOR Gate Predictions:")

for i in range(len(X)):
    prediction = activation(np.dot(X[i], weights) + bias)
    print(X[i], "=>", prediction)

    if prediction == y[i]:
        correct += 1

accuracy = correct / len(y) * 100

print("\nAccuracy:", accuracy, "%")
```

```
... Final Weights: [0.1 0.1]
Final Bias: -0.1
```

```
OR Gate Predictions:
```

```
[0 0] => 0
[0 1] => 1
[1 0] => 1
[1 1] => 1
```

4. XOR Classification Analysis

```
import numpy as np

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 1, 1, 0])

weights = np.zeros(2)
bias = 0

learning_rate = 0.1
epochs = 20

def activation(z):
    return 1 if z >= 0 else 0

for epoch in range(epochs):
    errors = 0

    for i in range(len(X)):
        prediction = activation(np.dot(X[i], weights) + bias)

        error = y[i] - prediction

        weights += learning_rate * error * X[i]
        bias += learning_rate * error

        if error != 0:
            errors += 1

    print("Epoch:", epoch + 1, "Errors:", errors)

print("\nFinal Weights:", weights)
print("Final Bias:", bias)

print("\nXOR Predictions:")

for i in range(len(X)):
    prediction = activation(np.dot(X[i], weights) + bias)

    print(X[i], "=>", prediction)

print("\nConclusion:")
print("A single-layer perceptron cannot correctly classify XOR.")
print("XOR is not linearly separable.")
print("A Multilayer Perceptron (MLP) is required.")
```

```
--- Epoch: 1 Errors: 3
Epoch: 2 Errors: 3
Epoch: 3 Errors: 4
Epoch: 4 Errors: 4
Epoch: 5 Errors: 4
Epoch: 6 Errors: 4
Epoch: 7 Errors: 4
Epoch: 8 Errors: 4
Epoch: 9 Errors: 4
Epoch: 10 Errors: 4
Epoch: 11 Errors: 4
Epoch: 12 Errors: 4
Epoch: 13 Errors: 4
Epoch: 14 Errors: 4
Epoch: 15 Errors: 4
Epoch: 16 Errors: 4
Epoch: 17 Errors: 4
Epoch: 18 Errors: 4
Epoch: 19 Errors: 4
Epoch: 20 Errors: 4

Final Weights: [-0.1  0. ]
Final Bias: 0.0

XOR Predictions:
[0 0] => 1
[0 1] => 1
[1 0] => 0
[1 1] => 0

Conclusion:
A single-layer perceptron cannot correctly classify XOR.
XOR is not linearly separable.
A Multilayer Perceptron (MLP) is required.
```

5.Gradient Descent Optimization

```
[5]
✓ 0s
import numpy as np
import matplotlib.pyplot as plt

def gradient_descent(learning_rate, iterations):

    w = 0
    cost_history = []

    for i in range(iterations):

        cost = (w - 3) ** 2

        gradient = 2 * (w - 3)

        w = w - learning_rate * gradient

        cost_history.append(cost)

    return w, cost_history

learning_rates = [0.01, 0.1, 0.5]

for lr in learning_rates:

    final_w, costs = gradient_descent(lr, 50)

    print("Learning Rate:", lr)
    print("Final w:", final_w)
    print("Final Cost:", costs[-1])
    print()

    plt.plot(costs, label="Learning Rate = " + str(lr))

plt.xlabel("Iterations")
plt.ylabel("Cost")
plt.title("Gradient Descent Convergence")
plt.legend()
plt.grid()
plt.show()

... Learning Rate: 0.01
Final w: 1.9074909597386487
Final Cost: 1.2427905071353385

Learning Rate: 0.1
Final w: 2.9999571825692186
Final Cost: 2.8645818417477395e-09

Learning Rate: 0.5
Final w: 3.0
Final Cost: 0.0
```

6. Batch GD, SGD, Mini-Batch GD

```
[7]
✓ 0s  import numpy as np
import time

X = np.random.rand(100, 2)
y = 3*X[:,0] + 5*X[:,1]

methods = ["Batch GD", "SGD", "Mini-Batch GD"]

for method in methods:

    start = time.time()

    weights = np.zeros(2)
    lr = 0.01

    for epoch in range(50):

        if method == "Batch GD":
            pred = X @ weights
            gradient = 2 * X.T @ (pred - y) / len(X)
            weights -= lr * gradient

        elif method == "SGD":
            for i in range(len(X)):
                error = X[i] @ weights - y[i]
                weights -= lr * 2 * error * X[i]

        else:
            for i in range(0, len(X), 10):
                Xb = X[i:i+10]
                yb = y[i:i+10]
                error = Xb @ weights - yb
                gradient = 2 * Xb.T @ error / len(Xb)
                weights -= lr * gradient

    print(method, "Time:", round(time.time()-start, 4))
```

▼ ... Batch GD Time: 0.0013
SGD Time: 0.0216
Mini-Batch GD Time: 0.0048

7. MLP for Iris Classification

```
import tensorflow as tf
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

iris = load_iris()

X = iris.data
y = OneHotEncoder(sparse_output=False).fit_transform(
    iris.target.reshape(-1, 1)
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, activation='relu'),
    tf.keras.layers.Dense(8, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=30, verbose=0)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", accuracy)
```

... Test Accuracy: 0.7333333492279053

8. Forward Propagation

```
[9]  import numpy as np

def sigmoid(x):
    return 1/(1+np.exp(-x))

X = np.array([[1, 2]])

W1 = np.array([[0.5, 0.2],
               [0.3, 0.4]])

b1 = np.array([[0.1, 0.2]])

W2 = np.array([[0.7],
               [0.6]])

b2 = 0.1

Z1 = X @ W1 + b1
A1 = sigmoid(Z1)

Z2 = A1 @ W2 + b2
A2 = sigmoid(Z2)

print("Hidden Weighted Sum:", Z1)
print("Hidden Activation:", A1)
print("Output Weighted Sum:", Z2)
print("Final Output:", A2)
```

... Hidden Weighted Sum: [[1.2 1.2]]
Hidden Activation: [[0.76852478 0.76852478]]
Output Weighted Sum: [[1.09908222]]
Final Output: [[0.7500881]]

9. Backpropagation

```
[10]
✓ 0s
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

W1 = np.random.randn(2,2)
W2 = np.random.randn(2,1)

lr = 0.5

def sigmoid(x):
    return 1/(1+np.exp(-x))

for epoch in range(5000):

    # Forward propagation
    A1 = sigmoid(X @ W1)
    output = sigmoid(A1 @ W2)

    # Error
    error = y - output

    # Backpropagation
    d2 = error * output * (1-output)
    d1 = (d2 @ W2.T) * A1 * (1-A1)

    # Update weights
    W2 += lr * A1.T @ d2
    W1 += lr * X.T @ d1

print("Predictions:")
print(np.round(output, 2))
```

... Predictions:
[[0.44]
[0.38]
[0.96]
[0.39]]

10. Curse of Dimensionality

```
14] from sklearn.datasets import make_classification
    from sklearn.model_selection import train_test_split
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.metrics import accuracy_score

    for features in [2, 10, 50, 100]:

        X, y = make_classification(
            n_samples=500,
            n_features=features,
            n_informative=2,
            n_redundant=0,
            n_repeated=0,
            random_state=42
        )

        X_train, X_test, y_train, y_test = train_test_split(
            X, y,
            test_size=0.2,
            random_state=42
        )

        model = RandomForestClassifier(
            n_estimators=50,
            random_state=42
        )

        model.fit(X_train, y_train)

        prediction = model.predict(X_test)

        accuracy = accuracy_score(y_test, prediction)

        print("Number of Features:", features)
        print("Accuracy:", round(accuracy, 2))
        print("-----")

*** Number of Features: 2
Accuracy: 0.93
-----
Number of Features: 10
Accuracy: 0.95
-----
Number of Features: 50
Accuracy: 0.85
-----
Number of Features: 100
Accuracy: 0.87
-----
```

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Content Accuracy	30	Highly accurate, indepth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides

Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness
------------------------	----	--	--	--	--

Faculty In-charge	Course Coordinator	Program Director
Signature: _____ Date: _____	Signature: _____ Date: _____	Signature: _____ Date: _____